

Prompt 1 - Mapping Core Features + UI

I want to build a clean, modern web app UI for a tool that scrapes top-performing tweets using a backend n8n automation. Please help me map out the core features, user flow, and UI layout for the front-end experience.

App Purpose:

The app lets users input a keyword and choose whether to scrape Top or Recent tweets. It then displays the following for a scraped post:

Tweet ID
URL
Content (the actual post)
Likes
Retweets
Replies
Quotes
Views
Date

The UI must:

- Accept a keyword + toggle between "Top" or "Recent" tweets
- Trigger an n8n webhook with that input
- Display results in a clean, scrollable layout
- Include performance stats, profile details, and tweet links

Prompt 2 - MVP User Journey Definition

I want to build all of the **core features you outlined** for MVP Phase One. Please help me define a **detailed user journey** that clearly walks through how a user would interact with the app from start to finish.

My goal is to:

- Fully understand what we're building in MVP Phase One
- Get a clear, step-by-step view of the user flow
- Use this journey to guide development and design

Please structure the user journey around:

1. **Initial experience** (landing on the app)
2. **Inputting a keyword and selecting a filter**
3. **Triggering the scrape and handling loading states**
4. **Displaying results and interacting with them**
5. **Optional edge cases** (no results, errors, mobile flow)

Keep it focused, realistic, and aligned with the feature set we defined earlier.

Prompt 3 - UI Development Plan for MVP

Based on everything we've discussed so far, please create a **detailed UI development plan** for the MVP of this product.

My goals are to:

- Understand all screens required for the MVP
- Get a component-level breakdown for each screen
- Map the flow between screens in a simple UI flow outline

Please include:

- 1. Screen-by-screen breakdown**
 - List every screen the MVP needs (e.g., homepage, input screen, results view, etc.)
 - For each screen, detail all UI components (e.g., input fields, buttons, cards, lists, headers, etc.)
 - Describe the purpose and logic of each section of the screen
- 2. UI Logic (No Styling)**
 - Do not include any visual styling (colors, fonts, shadows, etc.)
 - Focus purely on structure, components, and layout relationships
 - Use clear hierarchy (e.g., top nav > main input section > results area)
- 3. Brief UI Flow**
 - At the end, summarize the navigation flow between all screens in plain language (e.g., "User lands on Home → inputs keyword → views results → clicks link")

Keep the layout clean and modular, suitable for fast iteration and responsive web behavior.

Prompt 4 - Building The Back-End

I've completed the entire frontend based on the finalized UI development plan. Now I need your help drafting the **complete backend implementation plan** for the MVP.

Please follow these instructions:

- The plan should mirror the **MVP scope** and **UI flow** we've already defined
- Break it down into a **step-by-step checklist**, from backend setup to final integration
- Each step should be clear, specific, and actionable — I want to be able to follow the list line by line until the backend is complete
- Include any relevant API logic, data structure definitions, and integration points with the frontend and n8n
- Keep the plan **tech-stack neutral** unless I specify otherwise

My Goal:

Once I complete every step in the checklist, the full backend should be functional and ready to launch with the MVP.

Before you begin, let me know if you need clarification on:

- Backend tech stack
- Authentication (if any)
- How we'll call or structure the n8n automation
- Data storage preferences or external services used

Prompt 5 - Adding The UI

I've pasted a complete UI breakdown below (including layout, structure, colors, and fonts) from another web app.

Please rework this UI for **our product** based on the MVP we've defined in earlier messages.

What I want you to do:

- Keep the **general structure and layout** from the original breakdown
- Replace or adjust any parts that don't apply to our app
- Update all copy, sections, and component references to match the **purpose, features, and user flow of our MVP**
- Keep any relevant design elements (e.g., typography, spacing, layout style) that still fit

Your Output Should Include:

- A revised **section-by-section description** of the landing page
- All changes reflected through the lens of **our app's value prop**
- Clear alignment with our MVP scope and user journey

Only adapt what's needed — the goal is to reuse a great layout while making it relevant and specific to **our product**.



SCREEN-BY-SCREEN BREAKDOWN

1. Home Screen (Input Screen)

Purpose: Capture user input (keyword + filter) and trigger the scrape workflow

UI Component Breakdown:

- **Top Navigation (optional)**
 - App logo/title
 - Optional tagline or “About” link
- **Main Input Section**
 - **Heading** (e.g., “Scrape Top-Performing Tweets”)
 - **Subheading** (optional): Short description of app purpose
 - **Keyword Input Field**
 - Type: text
 - Placeholder: “Enter keyword...”
 - Required
 - **Filter Toggle**
 - Options: [Top] [Recent]
 - Toggle or radio group
 - Default: "Top"
 - **Scrape Button**
 - Label: “Scrape Tweets”
 - On click: disable inputs + trigger webhook

- **Loading State (Conditional)**
 - Spinner or message: “Scraping Tweets...”
 - Replaces results area until data is returned
 - **Error State (Conditional)**
 - Message block for error or network failure
 - Retry button (re-enables input section)
-

2. Results Screen (Post-Scrape View)

Purpose: Display tweets returned from backend in a clean, scrollable layout

UI Component Breakdown:

- **Top Section (Sticky or Repeated)**
 - Keyword recap: “Results for: ‘biohacking’”
 - Filter recap: “Top tweets” or “Recent tweets”
 - “New Search” button (optional - scrolls user to top input section)
- **Results Area (Scrollable)**
 - **Tweet Card (repeats per item)**
 - **Header Section**
 - Profile picture (optional)
 - Username + @handle
 - **Tweet Content**
 - Raw tweet text (line breaks preserved)
 - **Stats Row**

-  Likes
-  Retweets
-  Replies
-  Quotes
-  Views
- **Meta Info Section**
 -  Date
 -  Tweet ID (with optional copy button)
 -  View Tweet (link opens in new tab)
- **No Results State (Conditional)**
 - Message: “No tweets found for this keyword.”
 - Option to run a new search

UI LOGIC STRUCTURE (NO STYLING)

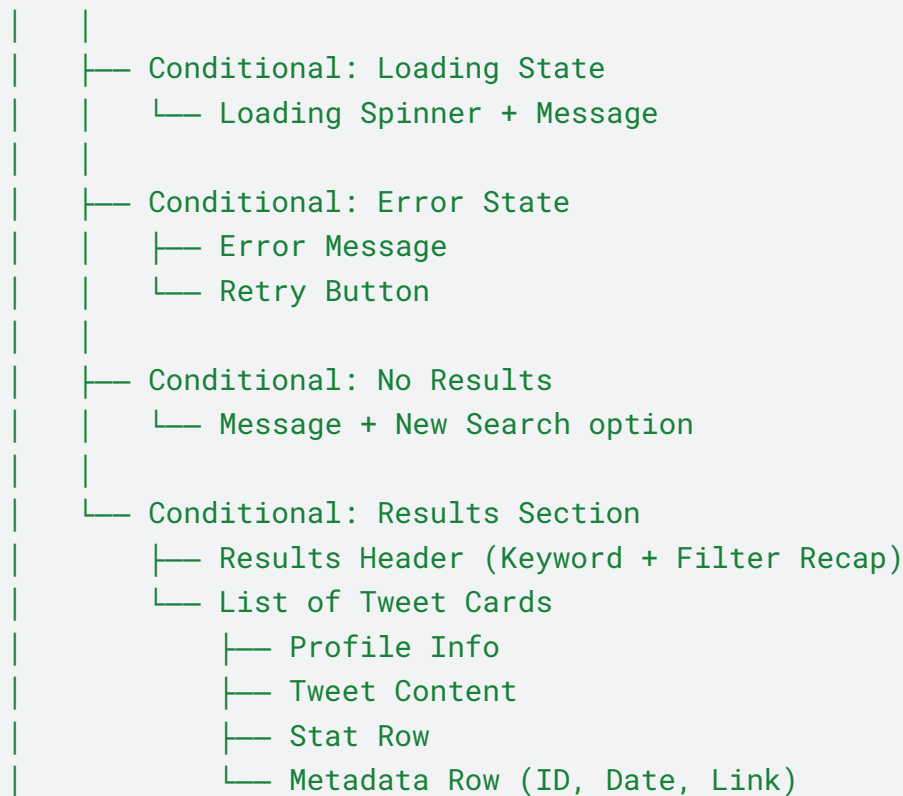
Unset

App Wrapper

```

├── Top Navigation (optional)
│   └── Logo / App Title / Links
│
├── Main Container
│   ├── Input Section
│   │   ├── Heading
│   │   ├── Subheading (optional)
│   │   ├── Keyword Input (text)
│   │   ├── Filter Toggle (Top / Recent)
│   │   └── Scrape Button

```

BRIEF UI FLOW

1. **User lands on Home Screen** → sees input UI immediately
2. **Enters keyword** + selects filter ("Top" or "Recent")
3. **Clicks Scrape Tweets** → loading state appears
4. **n8n webhook returns results**
 - If tweets found → Results Screen with tweet cards
 - If no tweets → No Results state
 - If error → Error message with Retry
5. **User browses tweet cards**

- Can click external tweet links
- Can scroll and/or copy tweet IDs

6. **User wants to try again** → scrolls up or clicks "New Search"